

CS 671 Fall 2025

HW1, Q3

Agreement 1: This assignment represents my own work. I have worked with the following people and/or LLMs: Brian Mason and Cursor/GPT5 -- though I do want to emphasize that the code even if LLM generated was based on my reasoning and understanding of the problem and LLMs were simply used to help me write the code in a more efficient manner.

Agreement 2: I understand that if I struggle with this assignment that I will reevaluate whether this is the correct class for me to take. I understand that the homework only gets harder.

Setup

```
In [1]: !pip install --upgrade treefarms
```

Requirement already satisfied: treefarms in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (0.2.4)

Requirement already satisfied: setuptools in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (75.8.0)

Requirement already satisfied: wheel in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (0.45.1)

Requirement already satisfied: attrs in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (24.3.0)

Requirement already satisfied: packaging>=20.9 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (24.2)

Requirement already satisfied: pandas in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (2.3.2)

Requirement already satisfied: scikit-learn in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (1.7.2)

Requirement already satisfied: sortedcontainers in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (2.4.0)

Requirement already satisfied: gmpy2 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (2.2.1)

Requirement already satisfied: matplotlib in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (3.10.6)

Requirement already satisfied: tqdm in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (4.67.1)

Requirement already satisfied: timbertrek in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (0.1.7)

Requirement already satisfied: editables==0.2 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (0.2)

Requirement already satisfied: contourpy>=1.0.1 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from matplotlib->treefarms) (1.3.2)

Requirement already satisfied: cycler>=0.10 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from matplotlib->treefarms) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from matplotlib->treefarms) (4.60.0)

Requirement already satisfied: kiwisolver>=1.3.1 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from matplotlib->treefarms) (1.4.9)

Requirement already satisfied: numpy>=1.23 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from matplotlib->treefarms) (2.0.1)

Requirement already satisfied: pillow>=8 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from matplotlib->treefarms) (11.3.0)

Requirement already satisfied: pyparsing>=2.3.1 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from matplotlib->treefarms) (3.2.4)

Requirement already satisfied: python-dateutil>=2.7 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from matplotlib->treefarms) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from pandas->treefarms) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from pandas->treefarms) (2025.1)

Requirement already satisfied: scipy>=1.8.0 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from scikit-learn->treefarms) (1.15.3)

Requirement already satisfied: joblib>=1.2.0 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from scikit-learn->treefarms) (1.5.2)

Requirement already satisfied: threadpoolctl>=3.1.0 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from scikit-learn->treefarms) (3.6.0)

Requirement already satisfied: ipython in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from timbertrek->treefarms) (8.30.0)

Requirement already satisfied: six>=1.5 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from python-dateutil>=2.7->matplotlib->treefarms) (1.16.0)

Requirement already satisfied: decorator in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from ipython->timbertrek->treefarms) (5.1.1)

Requirement already satisfied: jedi>=0.16 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from ipython->timbertrek->treefarms) (0.19.2)

Requirement already satisfied: matplotlib-inline in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from ipython->timbertrek->treefarms) (0.1.6)

Requirement already satisfied: prompt-toolkit<3.1.0,>=3.0.41 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from ipython->timbertrek->treefarms) (3.0.43)

Requirement already satisfied: pygments>=2.4.0 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from ipython->timbertrek->treefarms) (2.15.1)

Requirement already satisfied: stack-data in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from ipython->timbertrek->treefarms) (0.2.0)

Requirement already satisfied: traitlets>=5.13.0 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from ipython->timbertrek->treefarms) (5.14.3)

Requirement already satisfied: exceptiongroup in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from ipython->timbertrek->treefarms) (1.2.0)

Requirement already satisfied: typing-extensions>=4.6 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from ipython->timbertrek->treefarms) (4.12.2)

Requirement already satisfied: pexpect>4.3 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from ipython->timbertrek->treefarms) (4.8.0)

Requirement already satisfied: parso<0.9.0,>=0.8.4 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from jedi>=0.16->ipython->timbertrek->treefarms) (0.8.4)

Requirement already satisfied: ptyprocess>=0.5 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from pexpect>4.3->ipython->timbertrek->treefarms) (0.7.0)

Requirement already satisfied: wcwidth in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from prompt-toolkit<3.1.0,>=3.0.41->ipython->timbertrek->treefarms) (0.2.5)

Requirement already satisfied: executing in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from stack-data->ipython->timbertrek->treefarms) (0.8.3)

Requirement already satisfied: asttokens in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from stack-data->ipython->timbertrek->treefarms) (2.0.5)

Requirement already satisfied: pure-eval in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from stack-data->ipython->timbertrek->treefarms) (0.2.2)

```
In [2]: import pandas as pd
import os
from treefarms import TREEFARMS
import matplotlib.pyplot as plt
import numpy as np
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import f1_score
from sympy import to_dnf, simplify_logic
```

```
In [3]: def to_dnf_string(leaf) -> str:
        """Converts a leaf of a tree to a DNF expression
        for the path of variables to reach the leaf.
        """
        rules = leaf['rules']
        dnf = []
        for feature, rule in rules.items():
            if rule['type'] == 'Categorical':
                if rule['positive']:
                    dnf.append(f'({feature})')
                else:
                    dnf.append(f'(~{feature})')
        return ' & '.join(dnf)

def convert_to_dnf(tree):
    """Converts a tree to a DNF of its leaves.
    e.g. the tree (right splits are 1, left are 0)
        x3
      //  \
    x1    1
   //  \
  0     x2
   //  \
  1     0

    becomes the DNF

    (x1 & ~x2) | x3.
    """
    leaves = tree.__groups__()

    #otherwise, iterate through leaves
    pos_dnfs = []
    for leaf in leaves:
        if leaf['prediction'] == 1:
            pos_dnfs.append(to_dnf_string(leaf))

    # cover case where tree simplifies to True or False (like a stump, or a
    # tree with many leaves that all predict the same thing)
    if len(pos_dnfs) == 0: # if no leaf can ever predict positive:
        return False

    return to_dnf(' | '.join(pos_dnfs))

def get_simplified_dnf(tree):
    """Converts a tree to a simplified DNF of its leaves.
    """
    pos_dnf = convert_to_dnf(tree)
    simple_pos_dnf = simplify_logic(pos_dnf, form='dnf', force=True)
    return simple_pos_dnf

def print_dnf(dnf, features):
    """Print a DNF using the given feature names.
```

This is needed because sympy representations of DNFs use feature indices instead of their names. For example, the expression $(a \ \& \ \sim b) \mid c$ will be represented as $(\text{feature}_0 \ \& \ \sim \text{feature}_1) \mid \text{feature}_2$.

Use this to print DNF expressions in part d).

Parameters

```
dnf: A sympy expression representing a DNF.
features: A list of strings representing the feature names.
"""
dnf_str = str(dnf)
features = list(features)
for i in reversed(range(0, len(features))):
    dnf_str = dnf_str.replace(f'feature_{i}', features[i])
print(dnf_str)
```

```
In [4]: df = pd.read_csv('compas_hw1_q3.csv')
target = 'recidivate-within-two-years:1'
X = df.drop(target, axis=1)
y = df[target]
h = df.columns

# split the data into 70% train, 20% val, 10% test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.1, random_state=42)

print('train size:', X_train.shape[0]/X.shape[0])
print('val size:', X_val.shape[0]/X.shape[0])
print('test size:', X_test.shape[0]/X.shape[0])
```

```
train size: 0.6998696974084262
val size: 0.2000868683943825
test size: 0.10004343419719125
```

```
In [5]: # train TREEFARMS model
config = {
    "regularization": 0.01, # regularization penalizes the tree with more leaves
    "rashomon_bound_multiplier": 0.05, # rashomon bound multiplier indicates how much more complex the tree can be
    "depth_budget": 6
}

model = TREEFARMS(config)

model.fit(X_train, y_train)

models = [model[i] for i in range(model.get_tree_count())]
```

```

null
Finding Optimal Objective...
treefarm reported successful execution
training completed. Number of trees in the Rashomon set: 1603
{
  "false": {
    "false": {
      "complexity": 0.009999999776482582,
      "loss": 0.02027306519448757,
      "name": "recidivate-within-two-years:1",
      "prediction": 1
    },
    "feature": 7,
    "name": "juvenile-crimes:=0",
    "reference": 1,
    "relation": "==",
    "true": {
      "complexity": 0.009999999776482582,
      "loss": 0.21679767966270447,
      "name": "recidivate-within-two-years:1",
      "prediction": 0
    },
    "type": "integral"
  },
  "feature": 11,
  "model_objective": 0.3754695951938629,
  "name": "priors:>3",
  "reference": 1,
  "relation": "==",
  "true": {
    "complexity": 0.009999999776482582,
    "loss": 0.10839883983135223,
    "name": "recidivate-within-two-years:1",
    "prediction": 1
  },
  "type": "integral"
}
{
  "false": {
    "complexity": 0.009999999776482582,
    "loss": 0.0376499779522419,
    "name": "recidivate-within-two-years:1",
    "prediction": 1
  },
  "feature": 7,
  "model_objective": 0.3754695951938629,
  "name": "juvenile-crimes:=0",
  "reference": 1,
  "relation": "==",
  "true": {
    "false": {
      "complexity": 0.009999999776482582,
      "loss": 0.21679767966270447,
      "name": "recidivate-within-two-years:1",
      "prediction": 0
    },

```

```

    "feature": 11,
    "name": "priors:>3",
    "reference": 1,
    "relation": "==",
    "true": {
      "complexity": 0.009999999776482582,
      "loss": 0.09102192521095276,
      "name": "recidivate-within-two-years:1",
      "prediction": 1
    },
    "type": "integral"
  },
  "type": "integral"
}

```

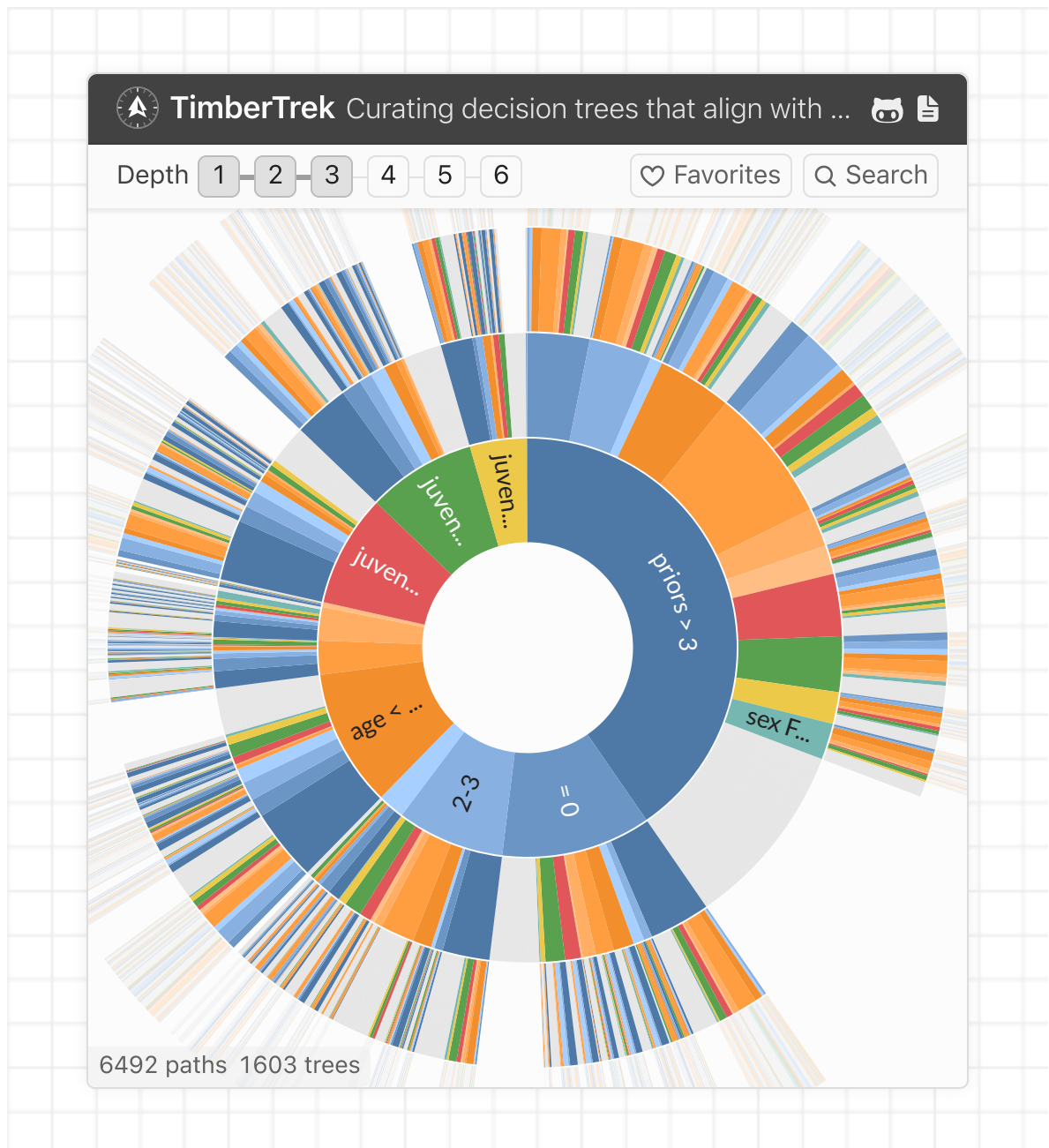
This is a TimberTrek representation of the Rashomon set of decision trees. Tree splits at different depths are represented in concentric rings, with rings further from the center representing deeper levels of the trees. Grey bars are individual models in the Rashomon set -- try clicking around to find models that you like, and play around with the search function to get rid of models using certain features!

In [6]: `model.visualize()`

```

Generating trie from 1603 trees: 100%|██████████| 1603/1603 [00:00<00:00, 62
944.99it/s]
Generating decision paths from 1603 trees: 100%|██████████| 1603/1603 [00:03
<00:00, 522.39it/s]

```



a) Exploring the Rashomon set

The Rashomon set fitting code is provided for you in the setup.

- i. Compute and store the number of leaves of each model in the Rashomon set, the train accuracy of each model, the validation accuracy of each model, and the validation f1 score of each model.

ii. Report the minimum and maximum of each item you computed in part i.

Q3 (a): Rashomon set model statistics

We compute leaves, train/val accuracy, and validation F1 for each model in the set, then report min/max.

```
In [7]: # Helper to compute accuracy

def accuracy(y_true, y_pred):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    return float((y_true == y_pred).mean())

# Collect per-model stats
stats = []
for m in models:
    # number of leaves (len of positive + negative groups)
    leaves = len(m.__groups__)
    # predictions
    yhat_tr = m.predict(X_train)
    yhat_va = m.predict(X_val)
    # metrics
    tr_acc = accuracy(y_train, yhat_tr)
    va_acc = accuracy(y_val, yhat_va)
    va_f1 = f1_score(y_val, yhat_va)
    stats.append({
        'leaves': leaves,
        'train_acc': tr_acc,
        'val_acc': va_acc,
        'val_f1': va_f1,
        'model': m,
    })

# Summary
leaves_list = [s['leaves'] for s in stats]
tr_list = [s['train_acc'] for s in stats]
va_list = [s['val_acc'] for s in stats]
va_f1_list = [s['val_f1'] for s in stats]

print(f"Leaves:      min={min(leaves_list)} max={max(leaves_list)}")
print(f"Train Acc: min={min(tr_list):.4f} max={max(tr_list):.4f}")
print(f"Val Acc:   min={min(va_list):.4f} max={max(va_list):.4f}")
print(f"Val F1:    min={min(va_f1_list):.4f} max={max(va_f1_list):.4f}")
```

```
Leaves:      min=2 max=6
Train Acc: min=0.6345 max=0.6721
Val Acc:   min=0.6353 max=0.6831
Val F1:    min=0.5607 max=0.6751
```

Q3 (b): Demographic Parity Difference for sex:Female

We compute DPD for each model, summarize min/max, and briefly interpret.

Interpretation: A DPD near 0 indicates similar average positive predictions across sexes.

A positive DPD means the model predicts positives more often when `sex:Female=0` (males) than `sex:Female=1` (females); negative means the reverse. The spread between min/max indicates variability in how models rely on sex.

b) Calculating Demographic Parity Difference

i. Fill in the provided function for computing demographic parity difference.

```
In [8]: def demographic_parity_difference(model, X, sensitive_feature):
        """Calculate the demographic parity difference of a model.

        Parameters
        -----
        model: A model implementing the predict function.
        X: A pandas dataframe of features.
        sensitive_feature: The name of the sensitive feature.

        Returns
        -----
        (float) The demographic parity difference.
        """
        # Predict on X
        y_hat = model.predict(X)
        s = X[sensitive_feature].values.astype(int)
        idx0 = (s == 0)
        idx1 = (s == 1)
        # Handle empty group edge cases gracefully
        p0 = y_hat[idx0].mean() if idx0.any() else 0.0
        p1 = y_hat[idx1].mean() if idx1.any() else 0.0
        return float(p0 - p1)
```

ii. Calculate the demographic parity difference of all of the models in the Rashomon set for the `sex:Female` feature. Report the minimum and maximum demographic parity difference inside the Rashomon set.

```
In [9]: # Compute DPD on validation set for sex:Female
        sensitive = 'sex:Female'
        for s in stats:
            s['val_dpd'] = demographic_parity_difference(s['model'], X_val, sensitive)
```

```
val_dpd_list = [s['val_dpd'] for s in stats]
print(f"DPD (sex:Female): min={min(val_dpd_list):.4f} max={max(val_dpd_list)}
```

```
DPD (sex:Female): min=0.1408 max=0.4461
```

iii. Interpret the minimum and maximum demographic parity differences inside the Rashomon set. What do the results imply about the variety of ways that the models in the Rashomon set decide if someone is likely to recidivate based on their sex?

The demographic parity difference among Rashomon-set models ranges from about 0.14 to 0.45. This means that, depending on the model, males are predicted to be between 14% and 45% more likely to recidivate than females on average. In other words, even though all these models are near-optimal in terms of accuracy, they vary widely in how strongly they use sex in making predictions. Some models exhibit relatively low disparities and treat males and females similarly, while others rely much more heavily on sex, leading to large differences in predictions. This highlights that the Rashomon set contains models with very different fairness properties, and shows why accuracy alone isn't enough to choose a model — we need to consider fairness metrics like demographic parity difference when deciding which model to deploy.

Q3 (c): Distributions and filtering

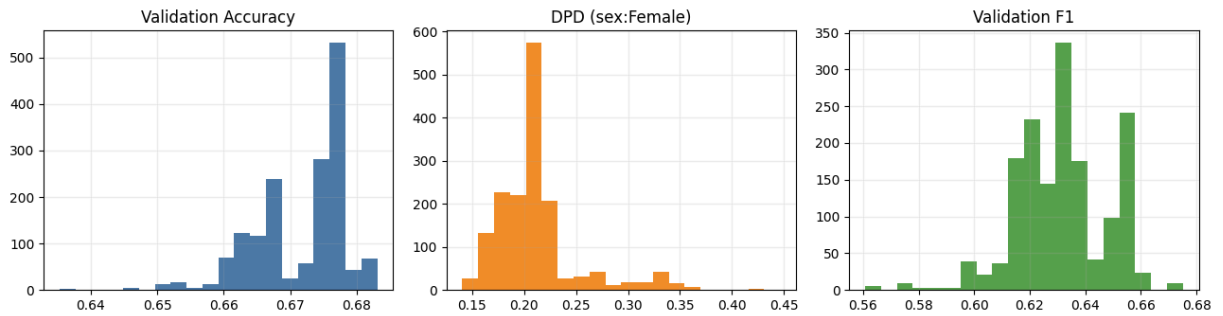
```
In [10]: # Histograms (20 bins)
vals_acc = [s['val_acc'] for s in stats]
vals_f1 = [s['val_f1'] for s in stats]
vals_dpd = [s['val_dpd'] for s in stats]

plt.figure(figsize=(13,3.5))
plt.subplot(1,3,1); plt.hist(vals_acc, bins=20, color='#4e79a7'); plt.title(
plt.subplot(1,3,2); plt.hist(vals_dpd, bins=20, color='#f28e2b'); plt.title(
plt.subplot(1,3,3); plt.hist(vals_f1, bins=20, color='#59a14f'); plt.title(
plt.tight_layout(); plt.show()

# Filters
filter1 = [s for s in stats if s['val_acc'] > 0.67]
print(f"After val_acc > 0.67: {len(filter1)} models")

filter2 = [s for s in filter1 if s['val_dpd'] < 0.2]
print(f"After DPD < 0.2: {len(filter2)} models")

filter3 = [s for s in filter2 if s['val_f1'] > 0.65]
print(f"After val_f1 > 0.65: {len(filter3)} models")
```



After `val_acc > 0.67`: 1004 models

After `DPD < 0.2`: 103 models

After `val_f1 > 0.65`: 34 models

c) Filtering the Rashomon set

i. Plot histograms, with 20 bins, of the validation accuracies, demographic parity differences w.r.t `sex:Female`, and validation f1 scores in the Rashomon set. We will inspect these histograms to choose cut points to filter the Rashomon set.

ii. Filter the Rashomon set down to models with validation accuracies > 0.67 . How many models are you left with?

After `val_acc > 0.67`: 1004 models

iii. Filter again down to models with validation demographic parity differences < 0.2 . How many models are you left with?

After `DPD < 0.2`: 103 models

iv. Filter once more to models with a validation f1-score > 0.65 . How many models are you left with?

After `val_f1 > 0.65`: 34 models

Q3 (d): Inspect DNFs of remaining models

```
In [11]: # to print just the DNFs
          ## Build set of simplified DNFs for the final filtered models
```

```

# remaining = [s['model'] for s in filter3]

# dnfs = []
# for m in remaining:
#     dnf = get_simplified_dnf(m)
#     dnfs.append(str(dnf))

# unique_dnfs = sorted(set(dnfs))
# print(f"Unique simplified DNFs: {len(unique_dnfs)}")
# for i, d in enumerate(unique_dnfs, 1):
#     print(f"[{i}] {d}")

# let's try to print both the DNFs and the Features

features = list(X.columns)

remaining = [{ 'model': s['model'], 'val_f1': s['val_f1'], 'val_dpd': s['val_

def name_dnf(dnf_expr):
    d = str(dnf_expr)
    for i in reversed(range(len(features))):
        d = d.replace(f'feature_{i}', features[i])
    return d

named_to_stats = {}
for r in remaining:
    dnf = get_simplified_dnf(r['model'])
    d_named = name_dnf(dnf)
    named_to_stats.setdefault(d_named, []).append((r['val_f1'], r['val_dpd']

print(f"Unique simplified DNFs (named): {len(named_to_stats)}")
# Order by best F1 within each DNF (desc)
for i, (d, lst) in enumerate(
    sorted(named_to_stats.items(), key=lambda kv: max(x[0] for x in kv[1]),
):
    f1s = [x[0] for x in lst]
    dpds = [x[1] for x in lst]
    print(f"[{i}] best F1={max(f1s):.4f} mean F1={np.mean(f1s):.4f} mean D
    print(f"      {d}")

```

```

Unique simplified DNFs (named): 9
[1] best F1=0.6544 mean F1=0.6544 mean DPD=0.1982 count=2
    age:<21 | (priors:2-3 & age:<26) | (priors:>3 & ~priors:2-3) | (~priors:
2-3 & ~juvenile-crimes:=0)
[2] best F1=0.6544 mean F1=0.6544 mean DPD=0.1982 count=2
    age:<21 | priors:>3 | (priors:2-3 & age:<26) | (~priors:2-3 & ~juvenile-
crimes:=0)
[3] best F1=0.6544 mean F1=0.6544 mean DPD=0.1982 count=6
    (priors:2-3 & age:<26) | (age:<21 & ~priors:2-3) | (priors:>3 & ~priors:
2-3) | (~priors:2-3 & ~juvenile-crimes:=0)
[4] best F1=0.6544 mean F1=0.6544 mean DPD=0.1982 count=2
    priors:>3 | (priors:2-3 & age:<26) | (age:<21 & ~priors:2-3) | (~priors:
2-3 & ~juvenile-crimes:=0)
[5] best F1=0.6534 mean F1=0.6534 mean DPD=0.1981 count=12
    age:<21 | priors:>3 | ~juvenile-crimes:=0 | (priors:2-3 & age:<26)
[6] best F1=0.6534 mean F1=0.6534 mean DPD=0.1981 count=2
    age:<21 | ~juvenile-crimes:=0 | (priors:2-3 & age:<26) | (priors:>3 & ~p
riors:2-3)
[7] best F1=0.6534 mean F1=0.6534 mean DPD=0.1981 count=2
    priors:>3 | ~juvenile-crimes:=0 | (priors:2-3 & age:<26) | (age:<21 & ~p
riors:2-3)
[8] best F1=0.6534 mean F1=0.6534 mean DPD=0.1981 count=4
    priors:>3 | ~juvenile-crimes:=0 | (age:<21 & age:<26) | (priors:2-3 & ag
e:<26)
[9] best F1=0.6534 mean F1=0.6534 mean DPD=0.1981 count=2
    ~juvenile-crimes:=0 | (priors:2-3 & age:<26) | (age:<21 & ~priors:2-3) |
(priors:>3 & ~priors:2-3)

```

d) Inspecting the remaining models

i. Use the provided helper code to examine the logical forms of the remaining models. How many unique simplified DNFs did we filter down to?

Only 9!

ii. Among the unique simplified DNF forms that you have remaining, use logical equivalence between expressions along with what you know about the features to identify how many of these are truly unique. (Hint: for example, you can't have `priors:>3` and `priors:=0`!)

Using the dataset semantics:

- **Priors buckets are one-hot:** exactly one of { `priors:=0` , `priors:=1` , `priors:2-3` , `priors:>3` } is true.
 $\Rightarrow (priors:>3 \ \& \ \sim priors:2-3) \equiv priors:>3$.

- **Age thresholds are nested:** `age:<21 ⇒ age:<26` .
- `~juvenile-crimes:=0` means “has ≥1 juvenile offense”.

and simplifying we get 2 unique DNFs:

- **[1]–[4]** become the same formula after removing redundancies
(e.g., the union of `(age:<21 & ~priors:2-3)` and `(priors:2-3 & age:<26)` is effectively `age:<21 v (priors:2-3 & age:<26)` ; and `(priors:>3 & ~priors:2-3)` is just `priors:>3` , etc.).
- **[5]–[9]** collapse to a second formula because each contains the unconditional disjunct `~juvenile-crimes:=0` (predict positive for any juvenile record), plus the same two “priors/age” patterns as above.

Hence the 9 DNFs reduce to **2 truly distinct logical functions**:

- **Class A (covers [1]–[4]):**

```
[ \text{age:<21} ;;\lor;; \text{priors:>3} ;;\lor;; (\text{priors:2-3} \land \text{age:<26})
;;\lor;; (\lnot\text{priors:2-3} \land \lnot\text{juvenile-crimes:=0}) ]
```

(i.e., uses juvenile history *conditionally* on priors not being 2–3.)

- **Class B (covers [5]–[9]):**

```
[ \text{age:<21} ;;\lor;; \text{priors:>3} ;;\lor;; (\text{priors:2-3} \land \text{age:<26})
;;\lor;; \lnot\text{juvenile-crimes:=0} ]
```

(i.e., triggers on *any* juvenile record regardless of priors.)

iii. The few remaining models are very similar to each other! Which one of them would you deploy? There are no wrong answers to this question.

Between these two models:

- **Class A ([1]–[4]):** best/mean F1 = 0.6544, mean DPD ≈ 0.1982.
- **Class B ([5]–[9]):** best/mean F1 = 0.6534, mean DPD ≈ 0.1981.

Accuracy and fairness metrics are almost identical, so the tie-breaker is the **logic**:

- **Class A** only uses juvenile history in context (requires “not priors:2–3,” with other priors disjuncts already explicit).

- **Class B** fires on *any* juvenile record (`~juvenile-crimes:=0`), regardless of priors—this is broader and more punitive.

Both families achieve nearly the same performance, but Class A is more measured: it doesn't penalize juvenile history in isolation, and instead considers it alongside adult priors. This aligns with a cautious, humane stance—acknowledging risk where multiple indicators agree (very young age, substantial adult priors, or a specific priors-and-age pattern), while avoiding blanket inferences from any juvenile record alone. Given the similar metrics, choosing the rule set that's less sweeping and more contextual feels fairer to the people affected.

This is a very cool problem, tysm for giving me the opportunity to work on it!